

SmartWatch Manager Project

{Predictive Maintenance and Energy Optimization Platform }

Problem Statement

In modern manufacturing, unexpected machinery downtime and inefficient energy consumption are two of the most critical factors driving up operational costs. Traditional maintenance strategies rely on reactive measures (fixing a machine after it breaks) or rigid scheduled maintenance (servicing equipment regardless of its actual wear and tear), both of which are economically inefficient.

Furthermore, industrial plants lack unified, granular visibility into how specific machines consume power in relation to their operational health. SmartWatch Manager addresses this by creating an end-to-end, edge-to-cloud ecosystem. It continuously tracks mechanical vibrations, temperature, and current consumption across manufacturing assets to predict equipment failure before it occurs, minimize downtime, and dynamically optimize factory energy consumption.

System Architecture

The system utilizes a decoupled, event-driven architecture designed to scale from a single machine to an entire factory floor.

1. Edge Layer	Microcontrollers interface with physical sensors, preprocess high-frequency raw telemetry, and publish data over a lightweight protocol.
2. Ingestion Layer	A central message broker ingests incoming streams, decoupling data production from consumption.
3. Processing & Storage	Lambda Architecture separating the Speed Layer (Hot Path via live WebSockets/Redis) and the Batch Layer (Cold Path via PostgreSQL RDBMS) for historical depth.

4. Application & Serving	A robust backend handles business logic, role-based access control, security, and hosts production ML model inference endpoints.
5. Presentation Layer	Responsive web and native mobile client applications consume the APIs and real-time streams to deliver actionable operational insights.

Technology Stack

Domain	Recommended Technologies
IIoT Edge	ESP32 or NodeMCU (ESP8266), C/C++ (Arduino IDE / ESP-IDF)
Data Streaming	Apache Kafka or Eclipse Mosquitto (MQTT) + Node-RED / Apache NiFi
Backend & APIs	Node.js (Express) or Python (FastAPI)
Database (RDBMS)	PostgreSQL (with TimescaleDB extension optional for hyper-scaling)
Data Science & ML	Python, Pandas, Scikit-Learn, TensorFlow/Keras, MLflow
Web Frontend	React.js, Tailwind CSS, Chart.js / Recharts
Mobile Frontend	React Native or Flutter (Cross-platform Android/iOS)
DevOps & Cloud	Docker, AWS (EC2, RDS) or Azure, GitHub Actions (CI/CD)

5. Hardware Requirements

- **Microcontrollers:** ESP32 Development Boards (preferred for dual-core processing and built-in Wi-Fi).
- **Vibration/Acceleration Sensor:** MPU6050 (3-axis accelerometer and gyroscope) to detect mechanical anomalies.
- **Temperature/Humidity Sensor:** DHT22 or DS18B20 (waterproof thermal probe for industrial environments).
- **Current/Power Sensor:** SCT-013 non-invasive AC current transformer sensor module.
- **Actuators & Infrastructure:** 5V Relay Modules, active buzzers, status LEDs, breadboards, jumper wires, and 5V power supplies.

6. Software Requirements

- **Development Environments:** VS Code (with PlatformIO or Arduino extension), Android Studio / Xcode (for mobile compilation).
- **Containerization & Orchestration:** Docker Desktop, Docker Compose configuration.
- **Database Management:** pgAdmin 4 (for administrative control over PostgreSQL instances).
- **API Testing & Evaluation:** Postman Client.
- **ML Development Stack:** Jupyter Notebooks / Google Colab for core exploratory data analysis and model packaging.

7. Data Pipeline Architecture

1. **Ingestion:** The ESP32 edge node samples physical sensors at specified frequencies (e.g., vibration at 100Hz, temperature at 0.1Hz). It wraps the payload into a compact JSON format and publishes it to a structured MQTT topic.
2. **Broker Routing:** The Mosquitto MQTT broker serves as the message gateway. An asynchronous consumer engine subscribes to the telemetry pipeline.
3. **Transformation & Validation:** The pipeline consumer handles data cleansing, parses timestamps, and structures records for database ingestion.
4. **Dual-Route Persistence (Lambda Architecture):**
 - **Live Hot Stream:** Pushes real-time operational thresholds straight to active WebSockets using Redis pub/sub mechanism.
 - **Historical Cold Stream:** Batches telemetry records safely and performs low-overhead bulk inserts into the PostgreSQL engine.

8. Database Design Overview

The normalized relational structural schema is designed for quick query optimization, data integrity, and cross-entity tracking:

- **COMPANIES & FACTORIES:** Multi-tenant foundations supporting modern scalability structures.
- **MACHINES:** Detailed equipment registry tracks asset types, runtime variables, and baseline profiles.
- **TELEMETRY_DATA:** A highly indexed time-stamped log table storing primary device telemetry records.
- **USERS:** System identities mapping access clearance via structured Role-Based Access Control (RBAC).
- **MAINTENANCE_LOGS & ALERTS_LOG:** Complete history records tracks human engineering actions and automated error flags.

9. Web Application Features

- **Role-Based Access Control (RBAC):** Tailored dashboard panels dividing simple system operator duties from administrative analytical oversight.
- **Real-time Factory Canvas:** Interactive digital plant overview maps that change color configurations instantly (Green/Yellow/Red) following live WebSocket triggers.
- **Historical Analytical Modules:** Deep data charting canvas rendering massive metrics tracking with zoom, downsampling, and custom filters.
- **ML Inference Configuration Panel:** A control screen dashboard providing model verification scores, Remaining Useful Life (RUL) metrics, and retraining execution triggers.

10. Mobile Application Features

- **Biometric Layer Security:** Fingerprint/FaceID user authentication mappings for mobile field personnel.
- **Push Alerts Notification Engine:** Low-latency alerts powered by Firebase Cloud Messaging (FCM) tracking critical failure predictions.
- **Acknowledge & Escalate Workflow:** Interactive operational flows permitting mobile response technicians to sign off or elevate priority levels in real-time.
- **On-the-go Local Diagnostics:** Quick monitoring view showing live telemetry feeds for physical validation inspections directly at the asset station.

11. Machine Learning Use Case

- **Core Objective:** Forecast precise Remaining Useful Life (RUL) parameters and classify incoming industrial failure forms.
- **Processing & Engineering:** Time-domain parameter computation (RMS, peak metrics) and Frequency-domain conversion utilizing Fast Fourier Transform (FFT) features over raw vibration streams.
- **Model Implementations:** Random Forest Regressor / LSTM pipelines for continuous forecasting alongside XGBoost for classification mapping.

- **Production Packaging:** Target models are serialized into standalone components and deployed within micro-contained FastAPI services.

12. Deployment Architecture

- **Isolation Strategy:** Each micro-service (Backend API, UI Frontend, Database, MQTT Broker, and ML Service) is completely decoupled inside separate Docker containers.
- **Orchestration Layout:** Managed systematically by a centralized Master Docker Compose file.
- **Traffic Management:** Secured and routed systematically using Nginx Reverse Proxy with automated SSL termination configurations.
- **Delivery Engine:** Standardized GitHub Actions integration mapping automated validation and deployments to remote cloud environments (AWS/Azure).

13. Learning Objectives

- Hardware-Software Interfacing and non-blocking real-time physical systems programming.
- Event-Driven Engineering architectures including asynchronous pipeline control and state management.
- Relational Database structures design and time-series search logic query optimization.
- Production Grade Machine Learning lifecycle execution stretching from raw features extraction up to micro-service deployment.
- Cloud-Native Software Engineering configurations leveraging Docker, reverse proxies, and continuous infrastructure orchestration.

14. Expected Project Deliverables

- Functional Physical IoT Node Prototype equipped with sensors inside an industrial-grade case enclosure.
- Production Backend Source Code complete with JWT security layers, API routes, and broker consumers.
- Cross-Platform UI Client Deployments comprising the Web Dashboard canvas and Mobile App configurations.
- Packaged ML Pipelines featuring validation work-logs, saved weights, and independent REST microservices.
- DevOps Orchestration Scripting encompassing standard Docker configurations and complete automated CI/CD definitions.
- Academic Thesis & Documentation structured systematically covering design specs, architecture charts, and analysis logs.

15. Estimated Implementation Timeline

- Months 1–2: Hardware Prototyping & Edge Ingestion Setup. Finalizing ESP32 circuit configurations and validating basic MQTT message schemas.
- Months 3–4: Core Backend Development & Relational DB Layout. Setting up PostgreSQL models, constraints, indices, and backend validation routes.
- Months 5–6: UI Client Integrations. Developing the React dashboard, cross-platform mobile apps, and mapping real-time WebSocket layers.
- Months 6–7: Machine Learning Lifecycle Engineering. Collecting streaming sets, designing feature transformations (FFT), training models, and packaging inference services.
- Month 8: System Orchestration, Automated Cloud Deployments, and Thesis Finalization.

16. Industry Relevance and Career Benefits

This system maps directly to modern industrial transformation paradigms (Industry 4.0). Completing this project qualifies team members across diverse professional domains:

- IoT & Embedded Systems Engineer: Solid understanding of physical computing boundaries, protocols, and data filtering at the edge.
- Full-Stack / Cloud Software Engineer: Proven ability to build highly distributed applications, implement multi-tenant databases, and configure production containers.
- Data Engineer: Practical proficiency managing data streams, building robust data layers, and orchestration.
- Machine Learning Engineer: Direct practical expertise executing real-world production data analytics modeling beyond basic isolated desktop environments.